

RCC Onboarding - Part 4

Apptainer containers for reproducible and I/O-efficient biomedical workflows

IKIM RCC documentation proposal

11 July 2026

Contents

Purpose and learning goals

Information that administrators must complete

Why RCC uses containers

What a container is and is not

Why Conda environments perform poorly on network storage

Apptainer and the SIF image model

Image, cache, temporary space, and project data

Trust, provenance, and immutable references

Running Apptainer manually

Bind mounts and data paths

GPU-enabled containers

Using Apptainer from Snakemake

Building and publishing images

Performance and I/O patterns

Troubleshooting

Completion checklist

References for maintainers

Purpose and learning goals

Part 4 introduces Apptainer, the container runtime used by RCC. The main operational motivation is to deploy complex biomedical software without placing a large Conda environment containing thousands of small files on shared network storage.

By the end of Part 4, you should be able to:

explain the difference between an Apptainer image, a running container, and project data;

explain why a single read-only SIF image is more suitable for shared HPC use than an unpacked Conda environment;

pull or use an approved image and inspect its identity;
 execute a command with `apptainer exec`;
 understand bind mounts and why outputs remain outside the image;
 use `--cleanenv` and approved GPU integration;
 declare containers in Snakemake rules; and
 distinguish image building from image execution.

Primary RCC container principle Use an immutable Apptainer SIF image for deployed software. Keep research data and durable outputs in approved project storage. Use node-local temporary storage for I/O-intensive temporary work.

Information that administrators must complete

Replace every value marked **ADMIN** before publication.

Supported Apptainer version: [ADMIN: insert and test]

Approved image registries: [ADMIN: list registries and authentication requirements]

RCC image store or project convention: [ADMIN: define where approved .sif files belong]

Apptainer cache policy: [ADMIN: document APPTAINER_CACHEDIR location and quota]

Apptainer temporary-build policy: [ADMIN: document APPTAINER_TMPDIR and whether users may build]

Build service or CI process: [ADMIN: define how production images are created and reviewed]

Allowed bind paths: [ADMIN: document automatic and user-defined binds]

Network access from submission and compute nodes: [ADMIN: document where apptainer pull works]

GPU flags and supported CUDA policy: [ADMIN: validate `--nv` and driver compatibility]

RCC Snakemake profile settings: [ADMIN: define apptainer-prefix, `--cleanenv`, and scratch behavior]

Container security and vulnerability-review policy: [ADMIN: insert policy]

Support contact: [ADMIN: insert RCC support address]

1. Why RCC uses containers

Biomedical software has deep dependency trees

A modern workflow may depend on Python, R, Java, C/C++ libraries, command-line tools, system libraries, reference-data clients, and exact versions of each component. Installing everything independently for every user leads to conflicts and makes old analyses difficult to reproduce.

Conda and Bioconda are excellent for resolving and describing software packages, especially during development. However, an installed Conda environment is a directory tree containing many small files: executables, Python modules, shared libraries, metadata, certificates, and package records.

On a laptop or local SSD this can work well. On shared network storage, thousands of jobs opening thousands of small environment files creates random and metadata-heavy I/O. Jobs may spend substantial time importing modules and loading libraries before scientific computation begins.

Containers package the runtime into an image

Apptainer packages the operating-system user space and installed software into a SIF image. The image is typically one read-only file. Multiple jobs can use the same image without modifying it.

The performance objective is not merely a faster `conda activate`. It is to replace a network directory tree containing many small files with a compact, immutable image that is efficient to distribute, cache, verify, and execute on HPC nodes.

2. What a container is and is not

A container is a packaged user-space environment

A container image can include:

- command-line programs;
- language runtimes;
- libraries;
- package metadata;
- default environment settings; and
- labels describing its origin.

The running container uses the host Linux kernel. It is not a complete virtual machine and does not emulate arbitrary hardware.

A container is not the research project

Do not place raw patient-derived data, final results, credentials, or the only copy of scripts inside an image. The image should contain software. Project data remains in approved host directories and becomes visible inside the container through bind mounts.

A container does not guarantee scientific validity

A container can reproduce software versions and operating-system libraries. It cannot correct an inappropriate reference genome, biased cohort, invalid statistical model, contaminated sample, or undocumented parameter.

A container is not automatically secure or trustworthy

Images can contain outdated or malicious software. Use approved sources, immutable references, checksums or signatures, and institutional review processes.

3. Why Conda environments perform poorly on network storage

Revisit large versus small I/O

A Conda environment might occupy several gigabytes across tens of thousands of files. Starting Python can require opening many modules and shared libraries. A workflow with hundreds of simultaneous jobs repeats these operations across the network filesystem.

This pattern involves:

- many file opens and closes;
- random reads from unrelated locations;
- directory lookups;
- metadata checks;
- symbolic-link resolution; and
- package-cache access.

The total bytes may be modest, but the operation count and latency are high.

Streaming versus random access

Streaming I/O reads large contiguous blocks in order. Network storage can optimise this pattern with read-ahead and large transfers.

Random I/O repeatedly accesses small, scattered blocks. Each access pays latency.

Large-file I/O moves substantial content per open operation.

Small-file I/O performs many metadata and open operations for little content.

A SIF image is a single file containing a compressed SquashFS filesystem. Software files are read as blocks from the mounted image rather than as an unpacked environment tree. This reduces many host-filesystem directory and metadata operations. The container's internal access pattern can still involve different blocks, so placement and caching remain relevant.

Containers complement rather than eliminate Conda

Conda remains useful to define dependencies. A development environment or lock file can be used to build a container. The deployed workflow then runs the immutable SIF rather than activating the unpacked environment on every compute node.

Snakemake can also generate a container definition from rule-specific Conda environments, providing a migration path from development environments to containerised deployment.

4. Apptainer and the SIF image model

Why Apptainer is used on HPC

Apptainer is designed for shared Linux systems and HPC environments. It can run containers without a long-lived user-controlled daemon and integrates with Slurm allocations, host filesystems, and HPC devices.

SIF: one immutable image file

SIF stands for Singularity Image Format. A .sif file is generally read-only. Read-only images:

can be shared safely by many simultaneous jobs;

are not modified by normal execution;

can be checksummed, signed, copied, and archived;

avoid the small-file tree of an unpacked environment; and

support compressed SquashFS storage.

Example image name:

containers/rnaseq-2.4.1_sha256-abc123.sif

Include a meaningful version and record the cryptographic digest separately. File names are for humans; checksums or signed identities provide stronger evidence.

Image versus running container

The image is a file at rest. A running container is a process using that image. Each Slurm job starts its own process context. The read-only image can be reused concurrently.

5. Image, cache, temporary space, and project data

These locations solve different problems.

Location	Contains	Persistent?	Recommended storage
Approved image store	Reviewed .sif images used by workflows	Yes	Project or RCC image storage
Apptainer cache	Downloaded OCI layers and generated SIF cache entries	Rebuildable	RCC-approved cache; avoid uncontrolled growth in home

Location	Contains	Persistent?	Recommended storage
Apptainer temporary directory	Temporary data during pull/build/conversion	No	Fast space with sufficient capacity
Node-local scratch	Per-job temporary scientific data or staged image	No	\$TMPDIR / \$SLURM_TMPDIR as configured
Project data	Inputs, scripts, provenance, durable results	Yes	Approved project storage

Cache policy matters

Apptainer's default cache is commonly under `$HOME/.apptainer/cache`. On RCC, this may be inappropriate if home is quota-limited or if many users duplicate large images. Administrators should define the supported policy.

The cache directory should be private to one user and reside on a filesystem that supports required operations. Do not point multiple users at one writable shared cache.

Pre-pull production images

Compute nodes may have restricted internet access. Repeatedly pulling an image at job start is slow and unreliable. Prefer:

- build or pull once in an approved environment;
- verify the image;
- place the SIF in an approved image store;
- reference the local SIF path or immutable registry digest; and
- optionally stage the SIF to node-local storage when RCC recommends it.

6. Trust, provenance, and immutable references

Tags can change

A reference such as `docker://organisation/tool:latest` is convenient but not reproducible. The content behind latest, and sometimes even a version tag, can change.

Prefer:

- an immutable digest such as an OCI sha256 digest;
- a reviewed local SIF with a recorded SHA-256 checksum;
- a signed SIF where institutional key management supports it; and
- a definition file or Dockerfile stored in version control.

Record image identity with the analysis

```
sha256sum containers/workflow.sif
apptainer inspect containers/workflow.sif
apptainer inspect --labels containers/workflow.sif
```

Store the checksum, source URI, build recipe commit, build date, and test results in project provenance.

Use only trusted containers

A container executes software with your user permissions and can access paths bound into it. Do not run an untrusted image against sensitive project storage.

7. Running Apptainer manually

Use a small approved image to learn the commands.

Check the installation

```
apptainer --version
```

Execute one command

```
apptainer exec containers/approved-tool.sif tool --version
```

exec starts the named command inside the container.

Run the image's default action

```
apptainer run containers/approved-tool.sif
```

run executes the image's runscript, if defined. For reproducible workflows, explicit exec commands are usually easier to review.

Open a diagnostic shell

```
apptainer shell --cleanenv containers/approved-tool.sif
```

Use interactive shells for inspection and debugging, not as an undocumented production workflow.

Clean the host environment

Apptainer normally passes much of the host environment into the container. This can accidentally introduce host PYTHONPATH, R_LIBS, or other variables.

Use:

```
apptainer exec --cleanenv containers/approved-tool.sif tool --version
```

RCC's Snakemake profile should normally apply --cleanenv, with documented exceptions.

8. Bind mounts and data paths

How data enters the container

A bind mount maps a host path to a path visible inside the container. Some paths, such as the current working directory and home directory, may be bound automatically depending on site configuration.

Explicit example:

```
apptainer exec --cleanenv \
  --bind /projects/example:/work \
  containers/approved-tool.sif \
  tool --input /work/data/input.fastq.gz \
  --output /work/results/output.txt
```

The output is written to the host project directory. It does not become part of the read-only image.

Bind the narrowest required paths

Avoid exposing unrelated project trees. Use read-only binds for immutable input when supported and writable binds only for intended output locations.

Example concepts:

```

/projects/cohort/data    -> /input  (read-only)
/projects/cohort/results -> /output (read-write)
$TMPDIR                 -> /scratch (read-write, temporary)

```

Exact syntax and allowed options must be validated for RCC.

Paths must be designed consistently

Containerised tools should not depend on one researcher's absolute home path. Use stable in-container paths and let the execution layer bind the correct host paths.

9. GPU-enabled containers

Slurm allocates the GPU; Apptainer exposes it

Two separate actions are required:

request a GPU from Slurm using RCC's supported syntax; and

run the container with NVIDIA integration, commonly `--nv`.

Example pattern:

```
#SBATCH --gres=gpu:1
```

```
apptainer exec --cleanenv --nv model.sif python predict.py
```

The host provides the kernel driver and core NVIDIA libraries. The container provides the compatible user-space application stack. CUDA compatibility depends on the host driver, GPU architecture, and the version used to build the application.

Verify inside the allocation

```
apptainer exec --nv model.sif nvidia-smi
```

A successful `nvidia-smi` does not prove that the scientific application uses the GPU. Monitor application utilisation and throughput as described in Part 3.

10. Using Apptainer from Snakemake

Declare the image with the rule

```
rule fastp_qc:
```

```
    input:
```

```
        r1="data/{sample}_R1.fastq.gz",
        r2="data/{sample}_R2.fastq.gz"
```

```
    output:
```

```
        r1="results/{sample}_R1.fastq.gz",
        r2="results/{sample}_R2.fastq.gz",
        html="results/{sample}.fastp.html"
```

```
    threads: 8
```

```
    resources:
```

```
        mem_mb=8000,
        runtime=60,
        disk_mb=50000
```

```
    container:
```

```
        "docker://quay.io/biocontainers/fastp@sha256:IMMUTABLE_DIGEST"
```

```
    shell:
```

```
        r"""
        fastp --thread {threads} \
```



```
--in1 {input.r1} --in2 {input.r2} \
--out1 {output.r1} --out2 {output.r2} \
--html {output.html}
"""
```

Replace the teaching digest with a verified real digest.

Enable Apptainer execution

Current Snakemake supports:

```
snakemake --software-deployment-method apptainer
```

or:

```
snakemake --sdm apptainer
```

The RCC profile should normally provide this option and approved arguments:

```
executor: slurm
software-deployment-method: apptainer
apptainer-args: "--cleanenv"
apptainer-prefix: "[ADMIN: approved image/cache path]"
```

The exact profile must be tested with the RCC Slurm executor plugin.

Combine containers with node-local scratch

Containers solve software-file I/O. They do not automatically solve scientific-data I/O. Continue to use:

compressed inputs;

\$TMPDIR for tool temporary files;

Snakemake shadow for suitable random-I/O rules;

benchmark: for measurement; and

project storage only for durable data.

One global image or rule-specific images

A **global image** is simple and can work well for a tightly controlled workflow.

Rule-specific images reduce image size and isolate dependencies but may increase the number of images.

Choose a design that balances review, caching, startup cost, and maintenance. Do not build a new image for every sample.

Containerised Conda workflows

Snakemake can generate an Apptainer definition from rule-specific Conda environments:

```
snakemake --containerize apptainer > workflow.def
```

This can help migrate a tested Conda-based workflow into a deployed image. The generated definition still requires review, building, vulnerability assessment, testing, and immutable publication.

11. Building and publishing images

Building is different from running

Running a reviewed SIF is a routine user activity. Building an image changes the software supply chain and may require network access, substantial temporary disk, elevated build mechanisms, or a controlled remote builder.

Production pattern:

version-controlled recipe

- > controlled build/CI
- > automated tests
- > security review/scanning
- > checksum/signature
- > approved image store
- > Snakemake workflow

Definition-file example

Bootstrap: docker
 From: mambaorg/micromamba:2.3.0

```
%labels
  org.opencontainers.image.title RCC teaching image
  org.opencontainers.image.version 1.0.0
```

```
%post
  # Install pinned software here using a reviewed lock file.
```

```
%test
  tool --version
```

This is a conceptual skeleton, not a production image. Pin base images by digest and preserve lock files.

Test the scientific interface

Container tests should check more than `tool --version`. Include:

- a tiny synthetic input;
- expected output files;
- expected exit status;
- key numerical or checksum assertions;
- CPU/GPU startup as applicable; and
- execution through the RCC Snakemake profile.

12. Performance and I/O patterns

Good pattern

one reviewed SIF image

- + compressed project inputs
- + node-local temporary work
- + durable project outputs
- + benchmark and provenance files

Avoid writable sandboxes for production

An Apptainer sandbox is an unpacked directory tree. It brings back the many-small-file behavior that SIF is intended to avoid and is harder to verify as one immutable object. Writable overlays also complicate concurrent use and provenance.

Use read-only SIF images for production. Use writable development mechanisms only in controlled build workflows.

Consider staging the image itself

A SIF is one file and is designed to run from shared storage. For a very large image used by many I/O-intensive jobs, RCC may recommend copying it once per node or job to local scratch. Measure startup and repeated-read behavior before adding this complexity.

Do not copy the same multi-gigabyte image independently for thousands of very short jobs if the copy costs more than execution. Job grouping or node-level caching may be better.

Keep images focused

A huge image containing every bioinformatics tool may be easy to start with but slow to distribute, difficult to review, and hard to update. Prefer coherent images aligned with workflow stages or validated software stacks.

Clean caches deliberately

Inspect and clean only your own cache according to policy:

apptainer cache list

apptainer cache clean `--dry-run`

Do not schedule uncontrolled cache deletion while workflows are using images.

13. Troubleshooting

apptainer: command not found

The software is not loaded or available on that host. Follow RCC's supported activation method. Do not install a private runtime without approval.

Image cannot be pulled

Possible causes include blocked compute-node network access, registry rate limits, authentication, an invalid URI, or insufficient cache space. Pre-pull through the approved process.

permission denied for output

Check the host directory permissions and bind mapping. The container runs as your user and does not grant access you do not already have.

Tool exists on the host but not in the container

The container has its own software environment. Add the tool to the image recipe or use a different approved image. Do not rely on accidental host PATH leakage.

Python or R imports unexpected host packages

Use `--cleanenv` and check automatic bind mounts. Remove host PYTHONPATH, R_LIBS, and related variables from the execution environment.

Container works manually but fails in Slurm

Compare host, working directory, binds, environment, scratch variables, GPU allocation, and network availability. Reproduce with a small Slurm job rather than debugging on the submission host.

GPU is not visible

Confirm that the job actually received a GPU, use `--nv`, and check compatibility. CUDA_VISIBLE_DEVICES is normally controlled by Slurm and should not be overwritten casually.

Startup is still slow

Measure image location, image size, shared-storage load, bind-path metadata, program imports, and scientific input staging. Containers reduce software-tree I/O but do not eliminate every source of latency.

14. Completion checklist

You can complete Part 4 when you can:

explain why RCC prefers immutable SIF images over unpacked network Conda environments;

distinguish streaming, random, large-file, and small-file I/O;

explain image, container process, cache, temporary storage, and project data;

run an approved command with `apptainer exec --cleanenv`;

inspect and checksum an image;

explain how bind mounts expose host data;

use a container directive in a Snakemake rule;

explain why an immutable digest is preferable to latest;

describe the separate build, review, publication, and execution stages; and

combine containers with the Part 3 node-local scratch and benchmarking patterns.

References for maintainers

Apptainer User Guide, SIF/OCI support, read-only images, bind paths, cache, environment, GPU support, `exec`, `pull`, and `build`.

Snakemake 9.23.1, Distribution and Reproducibility: `rule containers` and `--software-deployment-method apptainer`.

Snakemake 9.23.1, Command Line Interface: `apptainer-prefix`, `apptainer-args`, and software deployment.

Slurm GPU/GRES documentation.

All registry names, image digests, cache paths, build procedures, and GPU flags must be validated on the production RCC environment before publication.